LEARNING VECTOR QUANTIZATION (LVQ)

NAMA	: Ihsanta Zaki
KELAS	: 2 D4 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Riyanto Sigit ST, M.Kom, Ph.D
MATA KULIAH	: Sistem Cerdas
TANGGAL	: Selasa, 8 April 2025

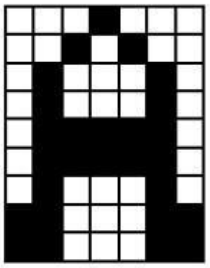
D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2025

Learning Vector Quantization (LVQ)

Tugas

- Akan dicoba untuk mengenali huruf A, B, atau H yang direpresentasikan dengan menggunakan kode 0 dan 1 pada matriks berukuran 9x7 seperti terlihat pada Gambar



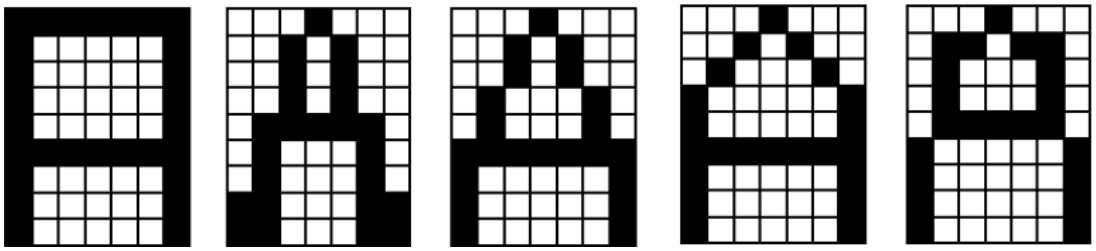
$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

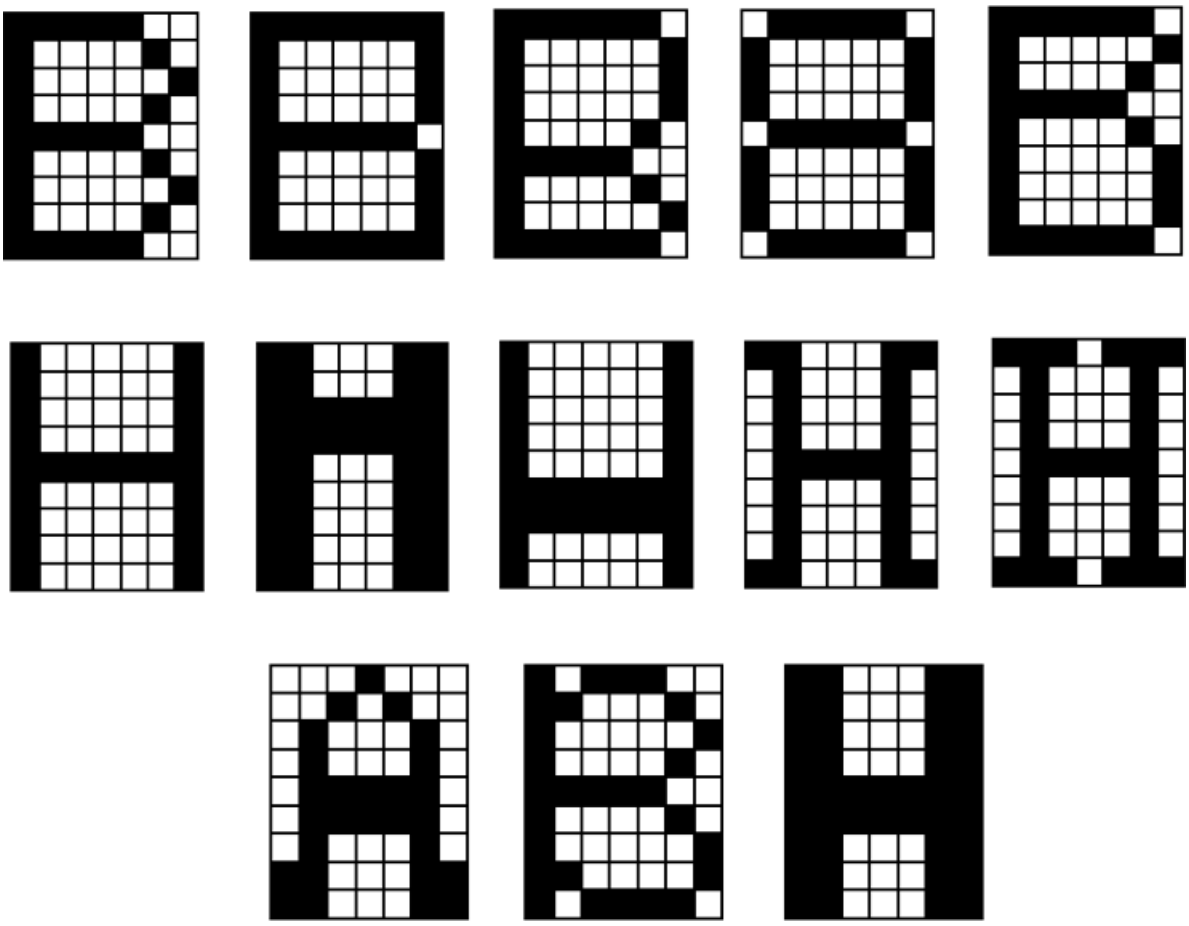
- Pada Gambar tersebut, kode 1 menunjukkan suatu kotak berwarna hitam, sedangkan kode 0 menunjukkan suatu kotak berwarna putih.

  - Untuk mempermudah dalam implementasi, matriks 9x7 tersebut dibawa ke bentuk vektor 63 kolom. Pada Gambar tersebut vektor yang bersesuaian adalah:

0 0 0 1 0 0 0 0 1 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 0 0 0 1 0 1 1 0 0 0 1 1 1 1 0 0 0 1 1

  - Misalkan ada 15 data yang akan dilatih, yaitu 5 data A, 5 data B, dan 5 data H, sebagai berikut





Program

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define INPUT_SIZE 63
#define NUM_CLASSES 3
#define NUM_SAMPLES 15
#define MAX_EPOCH 10

// Fungsi untuk menghitung jarak Euclidean
float euclidean_distance(float *a, float *b, int size) {
    float sum = 0.0;
    for (int i = 0; i < size; i++) {
        sum += (a[i] - b[i]) * (a[i] - b[i]);
    }
    return sqrt(sum);
}

int main() {
```

```
// Data training (huruf A, B, H)
float training_data[NUM_SAMPLES][INPUT_SIZE] = {
    // Data untuk huruf A (5 sampel)
    { 1,1,1,1,1,1,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1 },
    { 0,0,0,1,0,0,0, 0,0,1,0,1,0,0, 0,0,1,0,1,0,0, 0,0,1,0,1,0,0, 0,1,1,1,1,1,0 },
    { 0,0,0,1,0,0,0, 0,0,1,0,1,0,0, 0,0,1,0,1,0,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0 },
    { 0,0,0,1,0,0,0, 0,0,1,0,1,0,0, 0,1,0,0,0,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1 },
    { 0,0,0,1,0,0,0, 0,1,1,0,1,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0 },
    // Data untuk huruf B (5 sampel)
    { 1,1,1,1,1,0,0, 1,0,0,0,0,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,1,0, 1,1,1,1,1,0,0 },
    { 1,1,1,1,1,1,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0 },
    { 1,1,1,1,1,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,1,0 },
    { 0,1,1,1,1,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 0,1,1,1,1,1,0 },
    { 1,1,1,1,1,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,1,0, 1,1,1,1,1,0,0, 1,0,0,0,0,1,0 },
    // Data untuk huruf H (5 sampel)
    { 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,1 },
    { 1,1,0,0,0,1,1, 1,1,0,0,0,1,1, 1,1,1,1,1,1,1, 1,1,1,1,1,1,1, 1,1,0,0,0,1,1 },
    { 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,1,1,1,0,1 },
    { 1,1,0,0,0,1,1, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0 },
    { 1,1,1,0,1,1,1, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0 }
};

// Target kelas (1:A, 2:B, 3:H)
int target[NUM_SAMPLES] = { 1,1,1,1,1, 2,2,2,2,2, 3,3,3,3,3 };

// Inisialisasi bobot
float weights[NUM_CLASSES][INPUT_SIZE];
for (int i = 0; i < INPUT_SIZE; i++) {
    weights[0][i] = training_data[0][i]; // kelas A
    weights[1][i] = training_data[5][i]; // kelas B
    weights[2][i] = training_data[10][i]; // kelas H
}

float alpha = 0.1;
float alpha_reduction = 0.1;

// Proses training
for (int epoch = 0; epoch < MAX_EPOCH; epoch++) {
    printf("Epoch %d, learning rate: %f\n", epoch + 1, alpha);

    for (int sample = 0; sample < NUM_SAMPLES; sample++) {
        float distances[NUM_CLASSES];
        for (int c = 0; c < NUM_CLASSES; c++) {
            distances[c] = euclidean_distance(training_data[sample], weights[c],
```

```
INPUT_SIZE);
            printf("Distance to class %d: %f\n", c + 1, distances[c]);
        }

        int winner = 0;
        for (int c = 1; c < NUM_CLASSES; c++) {
            if (distances[c] < distances[winner]) {
                winner = c;
            }
        }
        printf("Sample %d: Winner is class %d\n", sample + 1, winner + 1);

        int target_class = target[sample] - 1;
        for (int i = 0; i < INPUT_SIZE; i++) {
            if (winner == target_class) {
                weights[winner][i] += alpha * (training_data[sample][i] - weights[winner][i]);
            } else {
                weights[winner][i] -= alpha * (training_data[sample][i] - weights[winner][i]);
            }
        }
    }

    // Mengurangi alpha setelah setiap epoch
    alpha -= alpha_reduction * alpha;
}

// Testing
float test_data[3][INPUT_SIZE] = {
    { 0,0,0,1,0,0,0, 0,0,1,0,1,0,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0 },
    { 1,0,1,1,1,0,0, 1,0,0,0,0,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,1,0, 1,1,1,1,1,0,0 },
    { 1,1,0,0,0,1,1, 1,1,0,0,0,1,1, 1,1,1,1,1,1,1, 1,1,1,1,1,1,1, 1,1,0,0,0,1,1 }
};

printf("\nTesting phase:\n");
for (int sample = 0; sample < 3; sample++) {
    printf("Testing sample %d:\n", sample + 1);
    float test_distances[NUM_CLASSES];
    for (int c = 0; c < NUM_CLASSES; c++) {
        test_distances[c] = euclidean_distance(test_data[sample], weights[c],
INPUT_SIZE);
        printf("Distance to class %d: %f\n", c + 1, test_distances[c]);
    }

    int predicted_class = 0;
```

```
        for (int c = 1; c < NUM_CLASSES; c++) {
            if (test_distances[c] < test_distances[predicted_class]) {
                predicted_class = c;
            }
        }
        printf("Hasil klasifikasi: %s\n",
                predicted_class == 0 ? "A" : (predicted_class == 1 ? "B" : "H"));
    }

    return 0;
}
```

Hasil

```
Epoch 1, learning rate: 0.100000
Distance to class 1: 0.000000
Distance to class 2: 3.316625
Distance to class 3: 3.162278
Sample 1: Winner is class 1
Distance to class 1: 5.000000
Distance to class 2: 4.242640
Distance to class 3: 4.123106
Sample 2: Winner is class 3
Distance to class 1: 4.690416
Distance to class 2: 4.358899
Distance to class 3: 4.813522
Sample 3: Winner is class 2
Distance to class 1: 3.741657
Distance to class 2: 4.667976
Distance to class 3: 4.239104
Sample 4: Winner is class 1
Distance to class 1: 4.933558
Distance to class 2: 4.334743
Distance to class 3: 4.665833
Sample 5: Winner is class 2
Distance to class 1: 3.246536
Distance to class 2: 0.822131
Distance to class 3: 3.488553
Sample 6: Winner is class 2
Distance to class 1: 2.477902
Distance to class 2: 2.850172
Distance to class 3: 2.563202
```

```
Testing phase:
Testing sample 1:
Distance to class 1: 2.499698
Distance to class 2: 3.950553
Distance to class 3: 3.473662
Hasil klasifikasi: A
Testing sample 2:
Distance to class 1: 3.204299
Distance to class 2: 2.136908
Distance to class 3: 3.005301
Hasil klasifikasi: B
Testing sample 3:
Distance to class 1: 4.056026
Distance to class 2: 4.105771
Distance to class 3: 2.944558
Hasil klasifikasi: H
```